Lab 14 — Hands-on lab: configuring the stack without rebuilding

Goal: create ConfigMaps from a manifest and from files, a Secret, inject them as variables and as mounted files into the API and into an unmodified official nginx, then watch what a change does — or does not do — to running Pods.

Prerequisites — Lab 13 completed: k3s, `localhost/api:1.0` in the cluster.

Files provided (`files/`) - `configmap-api.yaml` — `APP_MESSAGE` and `APP_PROFILE` for the API. - `secret-db.yaml` — the database password, as `stringData` . - `deployment-api.yaml` — the API: `envFrom` the ConfigMap, the Secret mounted under `/run/secrets/db` . - `deployment-web.yaml` — `nginx:alpine` as is, configured by two mounted ConfigMaps, plus the `web` and `api` Services. - `nginx-proxy.conf` , `index.html` — the files that will become ConfigMaps.

Step 1 — ConfigMaps, two ways

```
mkdir -p ~/labo-docker/14 && cd ~/labo-docker/14 && cp <path-to-the-lab>/files/* .
kubectl create namespace lab14 ; kubectl config set-context --current --namespace=lab14
kubectl apply -f configmap-api.yaml
kubectl create configmap web-nginx --from-file=default.conf=nginx-proxy.conf
kubectl create configmap web-html --from-file=index.html
kubectl get configmaps
kubectl get configmap api-config -o yaml | sed -n '/^data:/,/^kind/p'
kubectl describe configmap web-nginx | sed -n '/^Data/,/^$/p' | head -8
```

Observe four ConfigMaps (including `kube-root-ca.crt` , present in every namespace), `api-config` with its two keys, and `web-nginx` whose `default.conf` key holds the entire nginx file.

Explanation. A ConfigMap is a dictionary: a value can be a word or a twenty-line file. `--from-file=key=file` picks the key's name — here the name the file will carry inside the container.

Step 2 — A Secret, and what its "encryption" is worth

```
kubectl apply -f secret-db.yaml
kubectl get secret db-secret
kubectl get secret db-secret -o yaml | grep -A1 '^data:'
kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d ; echo
kubectl create secret generic demo --from-literal=token=abc --dry-run=client -o yaml | grep -A1
'^data:'
```

Observe `db-secret Opaque 1` , the value `POSTGRES_PASSWORD: UzNjcjN0LXBn` , then `S3cr3t-pg` returned by `base64 -d` ; and a manifest produced without creating anything, with `token: YWJj` .

Explanation. `stringData` is converted to base64 by the API server; anyone who can read the Secret can decode it. Protection comes from RBAC, encryption at rest, and never committing this YAML in plain text.

Step 3 — Injecting: variables and files

```
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s ; kubectl rollout status deployment/web --
timeout=90s
kubectl exec deploy/api -- env | grep -E 'APP_|DB_'
kubectl exec deploy/api -- ls -l /run/secrets/db/
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD ; echo
kubectl exec deploy/api -- wget -qO- http://localhost:8080/ ; echo
```

Observe `APP_MESSAGE=bonjour depuis une ConfigMap`, `APP_PROFILE=staging` (from `envFrom`) and `DB_PASSWORD_FILE=/run/secrets/db/POSTGRES_PASSWORD` — but **not** the password in the environment; a file `POSTGRES_PASSWORD -> ..data/POSTGRES_PASSWORD` mounted read-only; `S3cr3t-pg` when you open it; and the API answering with the ConfigMap's message.

```
kubectl exec deploy/web -- ls -l /etc/nginx/conf.d/ /usr/share/nginx/html/
kubectl port-forward svc/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/api/ ; echo ; kill %1
```

Observe inside the `web` Pod — the **official** `nginx:alpine` image — a `default.conf -> ..data/default.conf` and an `index.html -> ..data/index.html` coming from the ConfigMaps, and the full chain answering through the proxy.

Explanation. Variables for the API, files for nginx and for the secret. No more `web:1.0` image nor `hostPath`: the configuration is a cluster object, available on any node.

Step 4 — Modifying a ConfigMap

```
kubectl patch configmap api-config -p '{"data":{"APP_MESSAGE":"message modifie"}}'
sleep 5 ; kubectl exec deploy/api -- env | grep APP_MESSAGE
kubectl rollout restart deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Observe after the `patch`, still `bonjour depuis une ConfigMap` in the running Pods; after `rollout restart`, `message modifie`.

```
kubectl patch configmap web-html -p '{"data":{"index.html":"<h1>index modifie par la
ConfigMap</h1>\n"}}'
sleep 75 ; kubectl exec deploy/web -- cat /usr/share/nginx/html/index.html
```

Observe, with no restart at all, `<h1>index modifie par la ConfigMap</h1>` (allow one to two minutes depending on the kubelet's sync).

Explanation. A variable is copied into the process at creation; a ConfigMap volume is a link to a directory the kubelet swaps. For variables, `rollout restart`; for files, patience — and an application that re-reads the file.

Step 5 — A key that does not exist

```
kubectl patch deployment api --type=json -
p='[{"op":"add","path":"/spec/template/spec/containers/0/env/-","value":
{"name":"APP_PROFILE","valueFrom":{"configMapKeyRef":{"name":"api-config","key":"profil"}}}}]'
sleep 8 ; kubectl get pods | grep api
kubectl describe pods -l app=api | grep "couldn't find key" | tail -1
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observe a new Pod in `CreateContainerConfigError` next to the two old `Running` ones, and in its Events `Error: couldn't find key profil in ConfigMap lab14/api-config`; then the rollback.

Explanation. References are resolved when the container is created: a missing key blocks the Pod, and the lab 12 rollout keeps the old Pods in service.

Clean-up

```
kubectl delete namespace lab14
kubectl config set-context --current --namespace=default
kubectl get configmaps -A | grep -c lab14
```

Observe `0`. ConfigMaps, Secret, Deployments and Services went away with the namespace. The files in `~/labo-docker/14/` are the basis for lab 15, which adds the database.

What you must be able to state now

- `apply -f`, `--from-file`, `--from-literal`, `--dry-run=client -o yaml`: four ways to create ConfigMaps and Secrets.
- `stringData` becomes `UzNjcjN0LXBn`; `base64 -d` gives back `S3cr3t-pg`: not encryption.
- `envFrom` → `APP_MESSAGE` in `env`; `secret` volume → `/run/secrets/db/POSTGRES_PASSWORD`, absent from the environment.
- Official `nginx:alpine` configured by two mounted ConfigMaps: `default.conf -> ..data/default.conf`.
- Modified ConfigMap: variable unchanged until `rollout restart`, file updated by itself within one to two minutes.
- Missing key: `CreateContainerConfigError`, `couldn't find key profil`, old Pods intact.